

## TEMA 3: DISEÑO E IMPLEMENTACIÓN DE LA CAPA MODELO

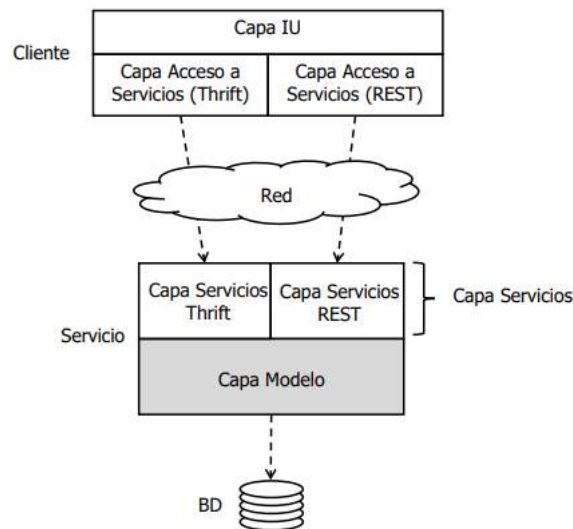
## OBJETIVO

- Aprender un **método** para desarrollar sistemáticamente la capa Modelo de una aplicación
  - El método se apoya en técnicas de diseño consolidadas
  - Las ideas generales del método son independientes de la tecnología
    - Aspectos concretos que dependen de las tecnologías usadas en la implementación
- Forma de aprendizaje
  - Estudiar el diseño de la capa Modelo que utiliza el servicio movies
  - El método es válido para el diseño de la capa Modelo de otro tipo de aplicaciones (ej. aplicaciones Web)

## MOVIES

- Corresponde al módulo **ws-movies** de los ejemplos de la asignatura
- Proporciona
  - Capa Modelo
    - Ofrece operaciones para gestionar información sobre películas (añadir, eliminar, etc.) y comprarlas
  - Capas de Servicios (Thrift y RESTful) → delegan en la capa Modelo
    - Exponen gran parte de la lógica del negocio, mediante una interfaz remota y + adaptada a las necesidades de sus clientes
  - Sencillo cliente de línea de comandos que puede invocar cualquiera de los servicios a través de una interfaz local común

## ARQUITECTURA GENERAL



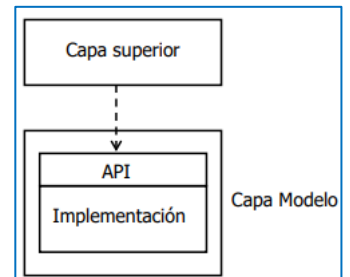
## ¿QUÉ OFRECE LA CAPA MODELO DEL SERVICIO MOVIES?

- **Casos de uso** para gestionar información sobre **películas**
  - Añadir
  - Actualizar
  - Eliminar una película si aún no tiene ninguna compra
  - Buscar una película por su clave
  - Buscar películas por palabras clave en el título
- Comprar películas
  - El usuario compra la película pagando con tarjeta de crédito
  - Cada compra genera una **venta**, que contiene una URL de un servicio de Video Streaming que permite visionar la película las veces que desee durante 2 días
  - Es posible recuperar la información de la venta a partir de su clave mientras la venta no expire

- Película
  - Identificador (clave numérica), título, duración, descripción, precios, fecha en alta en la BD
  - El identificador de la película es numérico y se debe generar automáticamente por el sistema
- Venta
  - Identificador de la venta (clave numérica), identificador de la película (clave foránea), identificador del usuario (clave foránea), fecha de expiración, número de tarjeta bancaria con la que se compró, precio al que se compró la película (puede variar), URL de streaming, fecha de la venta
  - El identificador de la venta es numérico y debe generar automáticamente por el sistema
- No se modelará información específica del usuario (nombre, apellidos, etc.)

## PRINCIPIOS DE DISEÑO

- La capa Modelo debe ofrecer una API que deba
  - **Invocar cada caso de uso de manera sencilla** → facilidad de uso para el desarrollador de la capa superior
  - **Ocultar detalles de implementación** → modificar la implementación sin que afecte a la capa superior



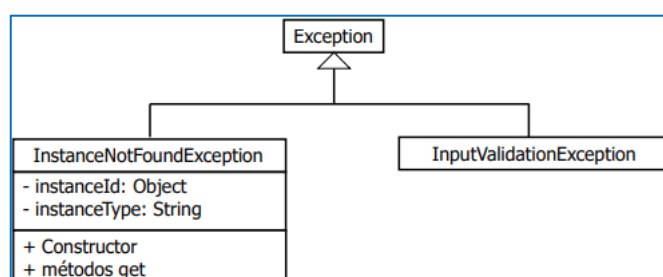
## MÉTODO DE DESARROLLO

### CONSIDERACIONES PREVIAS

- En **ws-javaexamples**
  - Módulo **ws-util**
    - Clases reusables
    - Paquete principal → **es.udc.es.util**
  - Módulo **ws-movies-model**
    - Capa Modelo de ejemplo Movies
    - Paquete principal → **es.udc.ws.movies.model**

### TRATAMIENTO DE EXCEPCIONES EN WS-JAVAEXAMPLES

- Errores lógicos
  - Representan (normalmente) errores del usuario final
  - Ej. actualizar una película que no existe, añadir una película con información de entrada en formato erróneo, etc.
  - En **ws-javaexamples** la capa Modelo notifica este tipo de errores lanzando excepciones de tipo “checked”
  - Normalmente, este tipo de excepciones se captura en la interfaz de usuario y muestran mensajes de error para que el usuario corrija los datos
  - **ws-util** → incluye excepciones para 2 tipos de errores “lógicos” muy frecuentes
    - **InstanceNotFoundException** → indica que se intenta hacer algo sobre un objeto inexistente (ej. actualizar/borrar una película que no existe)
    - **InputValidationException** → indica que se ha proporcionado información de entrada en formato erróneo (ej. campo numérico fuera de rango) → no es necesario conectarse a la base de datos para saber que algo está mal
    - Están definidas en **es.udc.ws.util.exceptions**



- Errores graves
  - Representan un error de infraestructura
  - Ej. base de datos caída, se intenta acceder a una tabla que no existe, etc.
  - Este tipo de situaciones se tratan lanzando **RuntimeException** (unchecked) → la excepción original encapsulada
  - Este tipo de excepciones se capturan de manera genérica en un único punto y se le indica al usuario que ha ocurrido un error interno

## PARÁMETROS DE CONFIGURACIÓN

- Leer parámetros de configuración (ej. parámetros de conexión a la BD) → **ws-util** proporciona la clase **ConfigurationParametersManager** (paquete **es.udc.ws.util.configuration**)
- Los parámetros se leen del fichero **ConfigurationParameters.properties** → debe estar disponible en el classpath
- Ej. formato de fichero

| ConfigurationParametersManager         |
|----------------------------------------|
| - parameters : Map<String, String>     |
| + getParameter(name : String) : String |

```
# MySQL.
url=jdbc:mysql://localhost/example?...
user=example
password=example
```

## [PASO 1] MODELAR LAS ENTIDADES (CLASES PERSISTENTES)

- Descripción de casos de uso → se deduce lo que hay que guardar en BD
  - Información películas
  - Información ventas
- Entidades
  - **es.udc.ws.movies.model.movie.Movie**
  - **es.udc.ws.movies.model.sale.Sale**
- En una app real las cantidades monetarias se modelan como **java.math.BigDecimal**
- Las entidades disponen de
  - Atributos privados para modelar el estado
  - Métodos públicos **get** para los atributos que se puedan leer
  - Métodos públicos **set** para los atributos que se puedan modificar
- Relación 1:N entre **Movie** y **Sale**
  - 1 película puede tener asociada múltiples ventas
  - El atributo **movieId** (clave foránea) en **Sale** mantiene la relación
- Atributos de tipo **LocalDateTime**
  - No interesa almacenar la parte fraccional de un segundo
  - En MySQL se almacenan en columnas de tipo **DATETIME** (no almacenan la parte fraccional)
  - Los constructores y los **setters** correspondientes se encargan de truncar la parte fraccional
    - Si no lo hiciesen, el valor del atributo de la entidad y el valor almacenado en el BD podría ser diferentes
    - Especialmente importante en las pruebas automatizadas → se comprueba la igualdad de fechas recuperadas de BD con fechas generadas en memoria
    - Ej. parte de un caso de prueba de un caso de uso que permite añadir películas, donde podrían ejecutarse los siguientes pasos
      - [1] Se crea una entidad de tipo **Movie** en memoria
      - [2] Se invoca al caso de uso que permite añadir una película, que establece el instante actual como valor de **creationDate** y almacena la entidad en BD
      - [3] Se recuperan de BD los valores almacenados en el paso anterior y se crea con ellos una nueva entidad de tipo **Movie** en memoria
      - [4] Se compara la entidad creada en el paso 1 (y modificada en el paso 2) con la creada en el paso 3

| Movie                          | Sale                             |
|--------------------------------|----------------------------------|
| - movieId : Long               | - saleId : Long                  |
| - title : String               | - movieId : Long                 |
| - runtime : short              | - userId : String                |
| - description : String         | - expirationDate : LocalDateTime |
| - price : float                | - creditCardNumber : String      |
| - creationDate : LocalDateTime | - price : float                  |
| + Constructores                | - movieUrl : String              |
| + métodos get/set              | - saleDate : LocalDateTime       |
|                                | + Constructores                  |
|                                | + métodos get/set                |

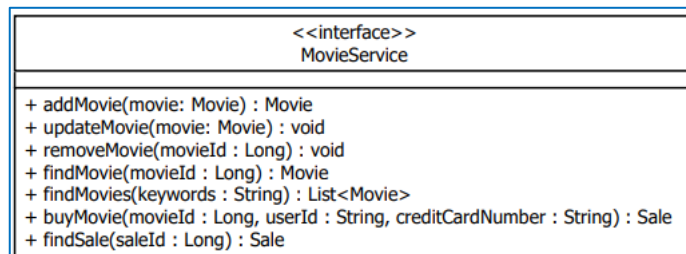
```

public class Movie {
    ...
    public Movie(Long movieId, String title, short runtime, String
        description, float price, LocalDateTime creationDate) {
        this(movieId, title, runtime, description, price);
        this.creationDate = (creationDate != null) ?
            creationDate.withNano(0) : null;
    }
    ...
    public void setCreationDate(LocalDateTime creationDate) {
        this.creationDate = (creationDate != null) ?
            creationDate.withNano(0) : null;
    }
    ...
}

```

## [PASO 2] DEFINIR UNA API SENCILLA PARA INVOCAR LOS CASOS DE USO

- Agrupar casos de uso de manera lógica
  - Distintas personas pueden pensar en distintos criterios para realizar la agrupación
- Definir una interfaz por cada grupo
  - Contiene un método por cada caso de uso
  - En cada método, los argumentos corresponden a los datos de entrada del caso de uso y el valor de retorno a los datos de salida
  - Cada una de las interfaces corresponde a la aplicación del patrón de diseño **Facade**
- **Por sencillez**, en Movies sólo se ha definido una fachada
  - **es.udc.ws.movies.model.movieservice.MovieService**
  - Convención de nombrado → **XxxService**
  - En un caso real se podría tener 2 fachadas
    - Casos de uso “de administración” → añadir, actualizar, eliminar, etc.
    - Casos de uso orientados al usuario comprador → buscar, comprar, etc.



```

public interface MovieService {

    public Movie addMovie(Movie movie) throws InputValidationException;

    public void updateMovie(Movie movie)
        throws InputValidationException, InstanceNotFoundException;

    public void removeMovie(Long movieId)
        throws InstanceNotFoundException, MovieNotRemovableException;

    public Movie findMovie(Long movieId)
        throws InstanceNotFoundException;

    public List<Movie> findMovies(String keywords);

    public Sale buyMovie(Long movieId, String userId,
        String creditCardNumber)
        throws InstanceNotFoundException, InputValidationException;

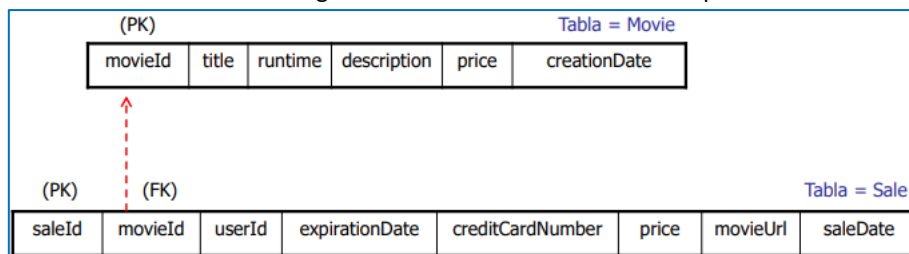
    public Sale findSale(Long saleId)
        throws InstanceNotFoundException, SaleExpirationException;
}

```

- La API del modelo se ha definido como una interfaz (**MovieService**)
  - Permite tener implementaciones alternativas
  - Movies proporciona la implementación **MovieServiceImpl**
  - En una fase temprana del desarrollo se podría haber proporcionado de manera rápida una implementación ficticia → ej. **MovieServiceMock**
    - No accede a BD
    - Los métodos no hacen nada o devuelven valores ficticios
  - Si en el equipo de desarrollo hay + de 1 persona, sería posible desarrollar la capa cliente de la capa Modelo mientras se implementa esta última (los DAOs y **MovieServiceImpl**) y en algún momento cambiar de **MovieServiceMock** a **MovieServiceImpl**
    - La capa cliente podría ser un servicio Web (como en Movies) o la interfaz gráfica (si la capa Modelo fuese local a la interfaz gráfica)
    - La capa cliente sólo trabaja contra la interfaz **MovieService**

### [PASO 3] PARA CADA ENTIDAD, CREAR UNA ABSTRACCIÓN QUE PERMITA GESTIONAR SU PERSISTENCIA

- Se usan las tablas para guardar películas y ventas
  - Cada instancia de una entidad se guarda en una fila de la tabla correspondiente



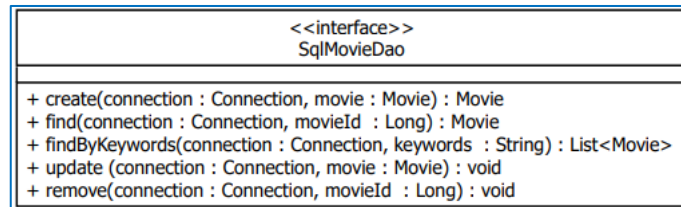
- Para implementar los casos de uso se puede necesitar
  - Operaciones básicas para cada entidad → insertar/leer/actualizar/eliminar una instancia
    - **CRUD** → Create-Read-Update-Delete
  - Operaciones avanzadas para cada entidad → búsquedas que devuelven una colección de instancias
    - Ej. películas cuyos títulos contienen un conjunto de palabras clave
- Ej. caso de uso “comprar una película”
  - Entrada → **movieId, userId, creditCardNumber**
  - Salida → **Sale**

```

Movie movie = << Leer de BD los datos de la película cuyo
    identificador es "movieId" >>;
LocalDateTime expirationDate = LocalDateTime.now().plusDays(2);
Sale sale = new Sale(movieId, userId, expirationDate,
    creditCardNumber, movie.getPrice(), <<getMovieUrl(movieId)>>,
    LocalDateTime.now());
<< Insertar "sale" en BD >>;
  
```

- Otros casos de uso también pueden necesitar estas operaciones de persistencia → leer, insertar, etc.
- Para facilitar la implementación de los casos de uso, se necesita una abstracción que permita gestionar la persistencia de cada entidad
- Sería **ideal** que la abstracción ocultase
  - BD concreta (MySQL, Redis, MongoDB, etc.)
  - Tipo de BD (relacional, clave-valor, orientadas a documentos, etc.)
  - Tecnología usada para acceder a la BD (uso directo de JDBC, API nativa, framework de alto nivel, etc.)
    - Si se cambia de BD o tecnología de acceso, no es necesario cambiar la implementación de los casos de uso
- Esta abstracción corresponde al patrón de diseño **DAO** (Data Access Object)
- Un DAO dispone de una interfaz y mínimo una clase de implementación
- DAOs en Movies
  - **es.udc.ws.movies.model.movie.SqlMovieDao**
  - **es.udc.ws.movies.model.sale.SqlSaleDao**
  - Convención de nombrado → **SqlXxxDao**

- Conceptualmente los DAOs corresponden a la “capa Acceso a Datos”



- Los DAOs que se definirán reciben la conexión (**java.sql.Connection**) para poder agrupar de manera sencilla varias operaciones de un mismo o distintos DAOs en una misma transacción
  - Los DAOs, consultan la BD concreta, pero no su tipo (relacional) ni la tecnología de acceso (JDBC) → por eso se nombran **SqlXxxDao**

```

public interface SqlMovieDao {

    public Movie create(Connection connection, Movie movie);

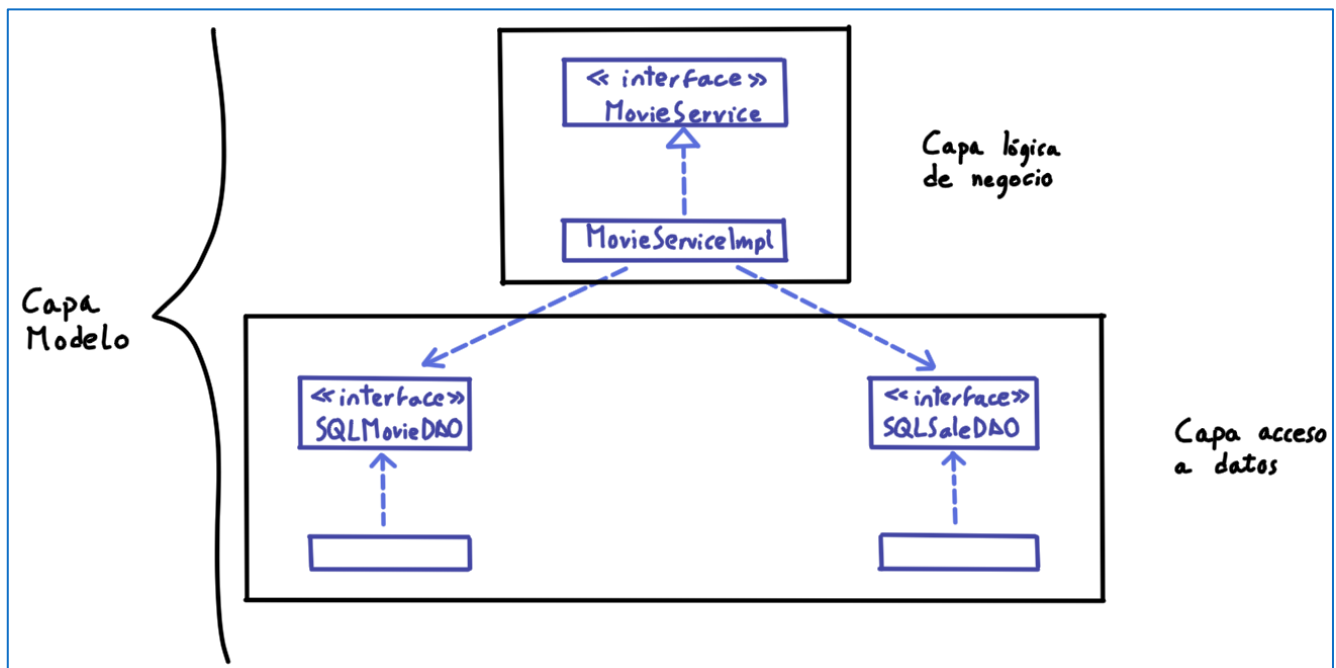
    public Movie find(Connection connection, Long movieId)
        throws InstanceNotFoundException;

    public List<Movie> findByKeywords(Connection connection,
        String keywords);

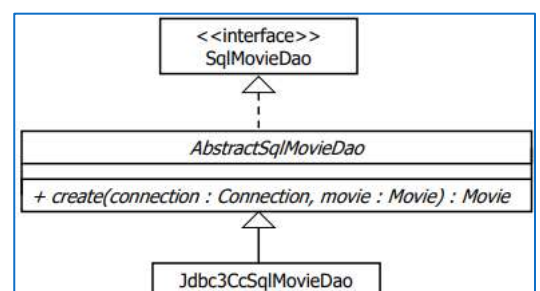
    public void update(Connection connection, Movie movie)
        throws InstanceNotFoundException;

    public void remove(Connection connection, Long movieId)
        throws InstanceNotFoundException;

}
    
```



- La implementación de los métodos de **SqlMovieDao** en **AbstractSqlMovieDao** es válida para cualquier BD relacional
- AbstractSqlMovieDao** deja sin implementar el método **create**
  - Este método tiene que crear dinámicamente la clave de la película
  - Existen múltiples estrategias para generar dinámicamente claves numéricas y no numéricas
  - Jdbc3CcSqlMovieDao** inserta una película en BD usando una estrategia particular para generar dinámicamente la clave numérica de la película





- Observaciones
  - Los objetos **PreparedStatement** son recursos
    - Implementan **AutoCloseable**
    - Cada método del DAO (ej. **find**) usa un bloque **try-with-resources** que cierra el objeto **PreparedStatement** inmediatamente cuando se termina de procesar la consulta → el **ResultSet** asociado también se libera en ese momento por la semántica de **close** en **PreparedStatement**
    - Las excepciones de tipo **SQLException** se relanzan como **RuntimeException** porque presentan errores de infraestructura

```
@Override
public List<Movie> findByKeywords(Connection connection,
    String keywords) {

    /* Create "queryString". */
    String[] words = keywords.split(" ");
    String queryString = "SELECT movieId, title, runtime, "
        + "description, price, creationDate FROM Movie";
    if (words.length > 0) {
        queryString += " WHERE";
        for (int i = 0; i < words.length; i++) {
            if (i > 0) {
                queryString += " AND";
            }
            queryString += " LOWER(title) LIKE LOWER(?)";
        }
    }
    queryString += " ORDER BY title";

    try (PreparedStatement preparedStatement =
        connection.prepareStatement(queryString)) {
```

```
        /* Fill "preparedStatement". */
        for (int i = 0; i < words.length; i++) {
            preparedStatement.setString(i + 1, "%" + words[i] + "%");
        }

        /* Execute query. */
        ResultSet resultSet = preparedStatement.executeQuery();

        /* Read movies. */
        List<Movie> movies = new ArrayList<Movie>();
        while (resultSet.next()) {
            int i = 1;
            Long movieId = new Long(resultSet.getLong(i++));
            // ...
            movies.add(new Movie(movieId, title, runtime,
                description, price, creationDate));
        }
        /* Return movies. */
        return movies;
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```

```
public abstract class AbstractSqlMovieDao implements SqlMovieDao {

    protected AbstractSqlMovieDao() {}

    @Override
    public Movie find(Connection connection, Long movieId)
        throws InstanceNotFoundException {

        /* Create "queryString". */
        String queryString = "SELECT title, runtime, description, " +
            "price, creationDate FROM Movie WHERE movieId = ?";

        try (PreparedStatement preparedStatement =
            connection.prepareStatement(queryString)) {

            /* Fill "preparedStatement". */
            int i = 1;
            preparedStatement.setLong(i++, movieId.longValue());

            /* Execute query. */
            ResultSet resultSet = preparedStatement.executeQuery();
            if (!resultSet.next()) {
                throw new InstanceNotFoundException(movieId,
                    Movie.class.getName());
            }

            /* Get results. */
            i = 1;
            String title = resultSet.getString(i++);
            short runtime = resultSet.getShort(i++);
            String description = resultSet.getString(i++);
            float price = resultSet.getFloat(i++);
            Timestamp creationDateAsTimestamp =
                resultSet.getTimestamp(i++);
            LocalDateTime creationDate =
                creationDateAsTimestamp.toLocalDateTime();
            /* Return movie. */
            return new Movie(movieId, title, runtime, description,
                price, creationDate);
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

```

@Override
public void update(Connection connection, Movie movie)
    throws InstanceNotFoundException {

    /* Create "queryString". */
    String queryString = "UPDATE Movie"
        + " SET title = ?, runtime = ?, description = ?, "
        + "price = ? WHERE movieId = ?";

    try (PreparedStatement preparedStatement =
        connection.prepareStatement(queryString)) {

        /* Fill "preparedStatement". */
        int i = 1;
        preparedStatement.setString(i++, movie.getTitle());
        preparedStatement.setShort(i++, movie.getRuntime());
        preparedStatement.setString(i++, movie.getDescription());
        preparedStatement.setFloat(i++, movie.getPrice());
        preparedStatement.setLong(i++, movie.getMovieId());

        /* Execute query. */
        int updatedRows = preparedStatement.executeUpdate();

        if (updatedRows == 0) {
            throw new InstanceNotFoundException(movie.getMovieId(),
                Movie.class.getName());
        }

    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}

```

```

@Override
public void remove(Connection connection, Long movieId)
    throws InstanceNotFoundException {

    /* Create "queryString". */
    String queryString = "DELETE FROM Movie WHERE" +
        " movieId = ?";

    try (PreparedStatement preparedStatement =
        connection.prepareStatement(queryString)) {

        /* Fill "preparedStatement". */
        int i = 1;
        preparedStatement.setLong(i++, movieId);

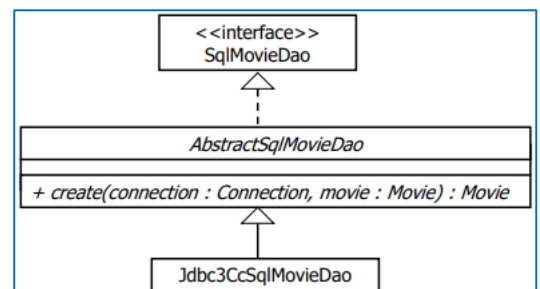
        /* Execute query. */
        int removedRows = preparedStatement.executeUpdate();

        if (removedRows == 0) {
            throw new InstanceNotFoundException(movieId,
                Movie.class.getName());
        }

    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}

```

- Generación dinámica de claves
  - El método **create** de **SqlMovieDao** de los ejemplos se implementó en **Jdbc3CcSqlMovieDao**, que inserta una película en BD usando una estrategia particular para generar dinámicamente la clave numérica de la película
  - Existen múltiples estrategias para la generación dinámica de claves numéricas
    - Algunas se apoyan directamente en mecanismos ofrecidos por la BD
    - Otras usan algoritmos para generar claves numéricas en memoria
  - Existen estrategias para genera dinámicamente claves de tipo **String**
    - Suelen construir cadenas de caracteres únicas usando una concatenación de varios campos → fecha, hora (en milisegundos), un número aleatorio, etc.
- Generación dinámica de claves numéricas soportadas por las BDs
  - Las BDs suelen diferir en cuanto al soporte que tienen para generar claves numéricas
  - Existen BDs que disponen de “generadores de identificadores numéricos”
    - contadores (secuencias) que se pueden leer e incrementar atómicamente
    - Ej. Oracle, PostgreSQL, etc.
    - En las BDs se puede generar el identificador (mediante una consulta especial, dependiente de la BD) e insertar la fila con el identificador generado





- Otras BDs disponen de “columnas contador”
  - Cada vez que se inserta una fila, se le asigna automáticamente un valor (el siguiente) a esa columna contador
  - No se puede consultar el valor antes de insertar la fila
  - Ej. MySQL, SQL Server, DB2, etc.
  - Es necesario
    - Insertar la fila (sin especificar un valor para la columna contador)
    - Leer el valor que se le ha asignado a la columna-contador lanzando una consulta especial
  - El paso 2 se puede implementar lanzando una consulta especial, dependiente de la BD, o usando la API de JDBC si se dispone de un driver JDBC 3.0 o superior
- **Jdbc3CcSqlMovieDao**
  - El objeto **PreparedStatement** se crea con una versión de **prepareStatement** que recibe adicionalmente el parámetro **Statement.RETURN\_GENERATED\_KEYS**
  - La consulta de inserción que se lanza con el **PreparedStatement** no especifica la clave
  - Cuando se ejecuta el método **getGeneratedKeys** de **PreparedStatement**, el driver de JDBC devuelve el identificador generado durante la inserción (en esa transacción)
    - La consulta devuelve un **ResultSet** con un único resultado → el identificador generado

```

public class Jdbc3CcSqlMovieDao extends AbstractSqlMovieDao {

    @Override
    public Movie create(Connection connection, Movie movie) {

        /* Create "queryString". */
        String queryString = "INSERT INTO Movie"
            + " (title, runtime, description, price, creationDate)"
            + " VALUES (?, ?, ?, ?, ?)";

        try (PreparedStatement preparedStatement =
            connection.prepareStatement(
                queryString, Statement.RETURN_GENERATED_KEYS)) {

            /* Fill "preparedStatement". */
            int i = 1;
            preparedStatement.setString(i++, movie.getTitle());
            // ...
            preparedStatement.setTimestamp(i++,
                Timestamp.valueOf(movie.getCreationDate()));

            /* Execute query. */
            preparedStatement.executeUpdate();

            /* Get generated identifier. */
            ResultSet resultSet = preparedStatement.getGeneratedKeys();

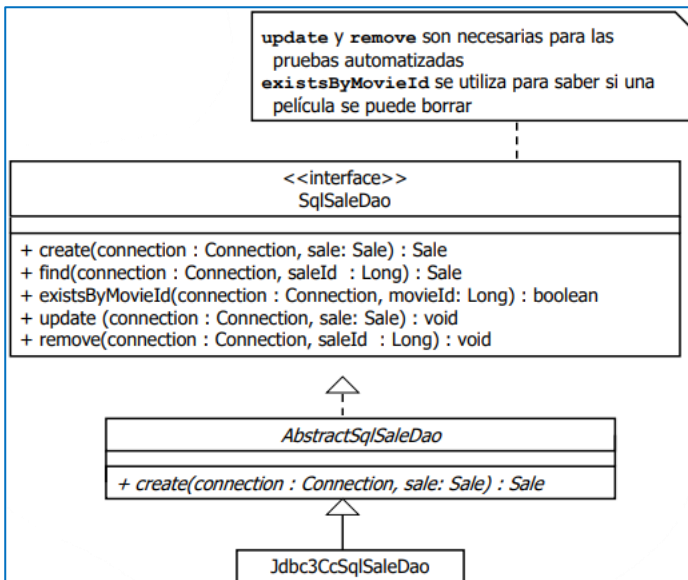
            if (!resultSet.next()) {
                throw new SQLException(
                    "JDBC driver did not return generated key.");
            }
            Long movieId = resultSet.getLong(1);

            /* Return movie. */
            return new Movie(movieId, movie.getTitle(),
                movie.getRuntime(), movie.getDescription(),
                movie.getPrice(), movie.getCreationDate());

        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}

```

- **SqlSaleDao**



- Ej. caso de uso “Comprar una película” en términos de DAOs
  - Entrada → **movieId, userId, creditCardNumber**
  - Salida → **Sale**

```

/* Get DAOs. */
SqlMovieDao movieDao = ...
SqlSaleDao saleDao = ...

/* Find Movie. */
Movie movie = movieDao.find(connection, movieId);

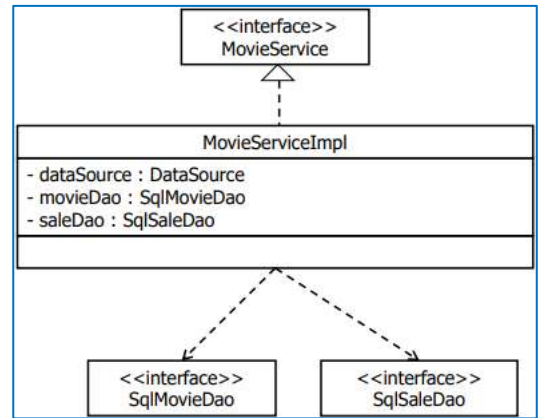
/* Create sale. */
LocalDateTime expirationDate = LocalDateTime.now().plusDays(2);
Sale sale = new Sale(movieId, userId, expirationDate,
    creditCardNumber, movie.getPrice(), <<getMovieUrl(movieId)>>,
    LocalDateTime.now());

/* Save sale. */
return saleDao.create(connection, sale);
  
```

Se explica más adelante cómo resolver este aspecto

## [PASO 4] IMPLEMENTAR LOS CASOS DE USO

- **MovieServiceImpl** implementa los casos de uso haciendo uso de los DAOs
  - Convención de nombrado → **XxxServiceImpl** (en el mismo paquete que la interfaz del servicio)
  - Conceptualmente, las clases de implementación de los servicios corresponden a la “capa Lógica de Negocio”
- Aspectos a tener en cuenta
  - Gestión de transacciones
  - Obtención de referencias a **DataSource**
  - Obtención de referencias a DAOs
  - Obtención de referencias al propio servicio **MovieService** desde la capa cliente
- Gestión de transacciones
  - Se seguirá un **enfoque sistemático**
  - Casos de uso que sólo ejecutan una operación de lectura contra la BD
    - Ej. **findMovie, findMovies**
    - Se usará el auto-commit y el nivel de aislamiento por defecto
  - Resto de casos de uso
    - Validar datos de entrada (si es necesario)
    - Empezar transacción
      - Establecer el nivel de aislamiento a **TRANSACTION\_SERIALIZABLE**
      - Deshabilitar el modo auto-commit de la conexión
    - Comprobar que es posible ejecutarlo en función de otros datos en BD (si es necesario)
      - Si no se puede → **commit** (finalizar la transacción) + excepción “checked”
    - Implementar la lógica de negocio usando los DAOs
    - Excepción correspondiente a un error grave
      - Si se produce → **rollback** + lanzar excepción
      - Si no se produce → **commit**



```

@Override
public Movie findMovie(Long movieId) throws InstanceNotFoundException {

    try (Connection connection = dataSource.getConnection()) {
        return movieDao.find(connection, movieId);
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
  
```

```

@Override
public List<Movie> findMovies(String keywords) {

    try (Connection connection = dataSource.getConnection()) {
        return movieDao.findByKeywords(connection, keywords);
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
  
```

```

@Override
public Movie addMovie(Movie movie) throws InputValidationException {

    validateMovie(movie);
    movie.setCreationDate(LocalDate.now());

    try (Connection connection = dataSource.getConnection()) {

        try {

            /* Prepare connection. */
            connection.setTransactionIsolation(
                Connection.TRANSACTION_SERIALIZABLE);
            connection.setAutoCommit(false);

            /* Do work. */
            Movie createdMovie = sqlMovieDao.create(connection, movie);

            /* Commit. */
            connection.commit();

            return createdMovie;

        } catch (SQLException e) {
            connection.rollback();
            throw new RuntimeException(e);
        } catch (RuntimeException|Error e) {
            connection.rollback();
            throw e;
        }

    } catch (SQLException e) {
        throw new RuntimeException(e);
    }

}

```

```

@Override
public Sale buyMovie(Long movieId, String userId,
    String creditCardNumber)
    throws InstanceNotFoundException, InputValidationException {

    PropertyValidator.validateCreditCard(creditCardNumber);

    try (Connection connection = dataSource.getConnection()) {

        try {

            /* Prepare connection. */
            connection.setTransactionIsolation(
                Connection.TRANSACTION_SERIALIZABLE);
            connection.setAutoCommit(false);

            /* Do work. */
            Movie movie = sqlMovieDao.find(connection, movieId);
            LocalDateTime expirationDate =
                LocalDateTime.now().plusDays(SALE_EXPIRATION_DAYS);
            Sale sale = sqlSaleDao.create(connection,
                new Sale(movieId, userId, expirationDate,
                    creditCardNumber, movie.getPrice(),
                    getMovieUrl(movieId), LocalDateTime.now()));

            /* Commit. */
            connection.commit();

            return sale;

        } catch (InstanceNotFoundException e) {
            connection.commit();
            throw e;
        } catch (SQLException e) {
            connection.rollback();
            throw new RuntimeException(e);
        } catch (RuntimeException|Error e) {
            connection.rollback();
            throw e;
        }

    } catch (SQLException e) {
        throw new RuntimeException(e);
    }

}

private static String getMovieUrl(Long movieId) {
    return BASE_URL + movieId + "/" + UUID.randomUUID().toString();
}

```

- No se muestran las implementaciones
  - **updateMovie** → misma estructura que **addMovie/buyMovie**
  - **removeMovie** → misma estructura que **addMovie/buyMovie**
    - Comprueba si existe alguna compra para la película que se quiere borrar
    - Si la hay lanza **MovieNotRemovableException**
  - **findSale** → misma estructura que **findMovie/findMovies**
    - Comprueba si la venta ha expirado
    - Si ha expirado lanza **SaleExpirationException**
- Validación de datos de entrada
  - La implementación de **addMovie** se apoya en el método privado **validateMovie** para validar el objeto **Movie** que recibe, que a su vez se apoya en la clase **PropertyValidator** (módulo **ws-util**, paquete **es.udc.ws.util.validation**)
  - La implementación de **buyMovie** también hace uso de **PropertyValidator** para validar el formato del número de la tarjeta de crédito
  - **PropertyValidator** es una sencilla clase utilidad que proporciona métodos para validar números enteros, reales, números simplificados de tarjetas de crédito, etc.
    - Si se detecta un error de validación, los métodos lanzan **InputValidationException**

```
private void validateMovie(Movie movie)
    throws InputValidationException {
    PropertyValidator.validateMandatoryString(
        "title", movie.getTitle());
    PropertyValidator.validateLong(
        "runtime", movie.getRuntime(), 0, MAX_RUNTIME);
    // ...
}
```

- Obtención de referencias a **DataSource**
  - En el constructor de **MovieServiceImpl** se inicializa el atributo **dataSource**

```
public MovieServiceImpl() {
    dataSource = // Estudiaremos ahora cómo resolver
                // este aspecto...
    movieDao = // ...
    saleDao = // ...
}
```

- El código tiene que ser capaz de obtener una referencia al **DataSource** de la aplicación en 2 situaciones distintas
  - [1] Cuando la capa Modelo se ejecuta dentro de una aplicación Web o un servicio Web
    - La aplicación o servicio Web se ejecuta dentro de un entorno (servidor de aplicaciones) que proporciona **DataSources** (típicamente con pool de conexiones)
  - [2] Cuando la capa Modelo es invocada por las pruebas de integración de la capa Modelo
    - Las pruebas se ejecutan en un entorno, generalmente, la línea de comandos (ej. **mvn test**) o un IDE, el cual NO proporciona **DataSources**
    - Proporcionar el **DataSource** a la capa Modelo es responsabilidad del desarrollador
- El módulo **ws-util** proporciona la clase **es.edc.ws.util.sql.DataSourceLocator** → permite que la implementación de los servicios de la capa Modelo (en el ejemplo **MovieServiceImpl**) pueda acceder a un **DataSource** abstrayéndose de las 2 situaciones en las que se ejecutará la capa Modelo
- **DataSourceLocator**

```
DataSourceLocator

public MovieServiceImpl() {
    dataSource = DataSourceLocator.getDataSource(
        MOVIE_DATA_SOURCE);
    movieDao = // ...
    saleDao = // ...
}
```

- El método **getDataSource** recibe el nombre del **DataSource** que se desea localizar (en el ejemplo **ws-javaexamples-ds**)

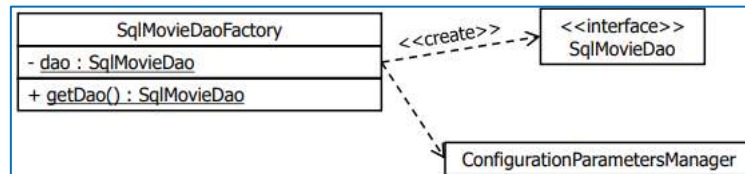


- Obtención de referencias a DAOs

- En el constructor de MovieServiceImpl se inicializan 2 atributos para los DAOs

```
public MovieServiceImpl() {
    dataSource = DataSourceLocator.getDataSource(
        MOVIE_DATA_SOURCE);
    movieDao = // Estudiaremos ahora cómo resolver
               // este aspecto...
    saleDao = // ...
}
```

- El patrón “Factory” permite obtener una referencia a un DAO sin conocer su clase de implementación



- **SqlMovieFactory**

- Lee el parámetro de configuración **SqlMovieDaoFactory.className** y crea una única instancia de esa clase (atributo estático **dao**) la cual siempre devuelve **getdao**
- La factoría trata al DAO como un Singleton, ya que es un objeto que no tiene estado, por lo tanto puede ser usado trivialmente por múltiples threads
- Si se desea usar otra estrategia de generación de identificadores numéricos o se cambia a una BD en la que el DAO especificado no sea válido, basta con proporcionar una nueva implementación del DAO y modificar el fichero de configuración

- En **ws-movies-model/src/main/resources/ConfigurationParameters.properties**

- En el constructor **MovieServiceImpl**

```
public MovieServiceImpl() {
    dataSource = DataSourceLocator.getDataSource(
        MOVIE_DATA_SOURCE);
    movieDao = SqlMovieDaoFactory.getDao();
    saleDao = SqlSaleDaoFactory.getDao();
}
```

- Convención de nombrado → **SqlXxxDaoFactory** (en el mismo paquete que la entidad)

```
public class SqlMovieDaoFactory {

    private final static String CLASS_NAME_PARAMETER =
        "SqlMovieDaoFactory.className";

    private static SqlMovieDao dao = null;

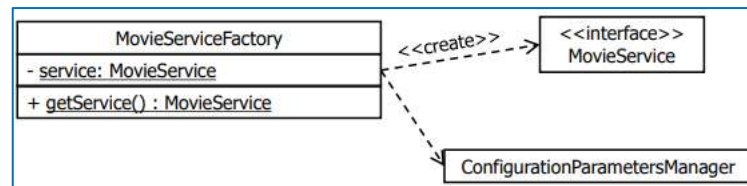
    private SqlMovieDaoFactory() {
    }

    @SuppressWarnings("rawtypes")
    private static SqlMovieDao getInstance() {
        try {
            String daoClassName = ConfigurationParametersManager.
                getParameter(CLASS_NAME_PARAMETER);
            Class daoClass = Class.forName(daoClassName);
            return (SqlMovieDao) daoClass.getDeclaredConstructor().
                newInstance();
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }

    public synchronized static SqlMovieDao getDao() {

        if (dao == null) {
            dao = getInstance();
        }
        return dao;
    }
}
```

- Obtención de referencias a MovieService
  - Se proporciona una factoría MovieServiceFactory que funciona de manera similar a las factorías de los DAOs
    - Permite que la capa cliente del modelo no esté acoplada a la clase de implementación de la interfaz MovieService
    - Convención de nombrado → XxxServiceFactory (en el mismo paquete que la interfaz del servicio)
  - La factoría trata al servicio como un Singleton porque no mantiene estado



- En ws-movies-model/src/main/resources/ConfigurationParameters.properties
- En una etapa temprana del desarrollo se podría especificar **MovieServiceMock** en el parámetro de configuración
  - Esto para ir desarrollando paralelamente la capa cliente y la capa Modelo
  - Posteriormente se especifica la clase de implementación real